

```
# =====
# Prerequisite: Environment Setup
# Covers Experiment No: 1 & 2
# Objective: Install Qiskit, configure the Python environment,
# and verify the installation of the quantum programming framework.
# =====
```

```
!pip install qiskit qiskit-aer pylatexenc
```

```
import qiskit
from qiskit_aer import AerSimulator
```

```
print("Qiskit version:", qiskit.__version__)
print("AerSimulator is ready for use!")
```

```
Collecting qiskit
  Downloading qiskit-2.3.1-cp310-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (12 kB)
Collecting qiskit-aer
  Downloading qiskit_aer-0.17.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (8.3 kB)
Collecting pylatexenc
  Downloading pylatexenc-2.10.tar.gz (162 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 162.6/162.6 kB 5.8 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Collecting rustworkx<=0.15.0 (from qiskit)
  Downloading rustworkx-0.17.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (10 kB)
Requirement already satisfied: numpy<3,>=1.21 in /usr/local/lib/python3.12/dist-packages (from qiskit) (2.0.2)
Requirement already satisfied: scipy>=1.5 in /usr/local/lib/python3.12/dist-packages (from qiskit) (1.16.3)
Requirement already satisfied: dill>=0.3 in /usr/local/lib/python3.12/dist-packages (from qiskit) (0.3.8)
Collecting stevedore>=3.0.0 (from qiskit)
  Downloading stevedore-5.7.0-py3-none-any.whl.metadata (2.4 kB)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from qiskit) (4.15.0)
Requirement already satisfied: psutil>=5 in /usr/local/lib/python3.12/dist-packages (from qiskit-aer) (5.9.5)
Requirement already satisfied: python-dateutil>=2.8.0 in /usr/local/lib/python3.12/dist-packages (from qiskit-aer) (2.9.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.0->qiskit-aer) (1.17.0)
  Downloading qiskit-2.3.1-cp310-abi3-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (8.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.8/8.8 MB 56.6 MB/s eta 0:00:00
  Downloading qiskit_aer-0.17.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.4 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.4/12.4 MB 65.5 MB/s eta 0:00:00
  Downloading rustworkx-0.17.1-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.2 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.2/2.2 MB 47.6 MB/s eta 0:00:00
  Downloading stevedore-5.7.0-py3-none-any.whl (54 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 54.5/54.5 kB 3.8 MB/s eta 0:00:00
Building wheels for collected packages: pylatexenc
  Building wheel for pylatexenc (setup.py) ... done
  Created wheel for pylatexenc: filename=pylatexenc-2.10-py3-none-any.whl size=136817 sha256=c3718c1dab878ae672b6
  Stored in directory: /root/.cache/pip/wheels/06/3e/78/fa1588c1ae991bbfd814af2bcac6cef7a178beee1939180d46
Successfully built pylatexenc
Installing collected packages: pylatexenc, stevedore, rustworkx, qiskit, qiskit-aer
Successfully installed pylatexenc-2.10 qiskit-2.3.1 qiskit-aer-0.17.2 rustworkx-0.17.1 stevedore-5.7.0
Qiskit version: 2.3.1
AerSimulator is ready for use!
```

```
# =====
# Experiment No: 3
# Objective: Understand qubits, classical bits, quantum registers,
# and classical registers by explicitly creating them in Qiskit.
# =====

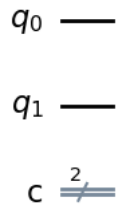
from qiskit import QuantumRegister, ClassicalRegister, QuantumCircuit

# Explicitly create a Quantum Register (to hold qubits)
# and a Classical Register (to hold measurement results)
q_reg = QuantumRegister(2, name='q')
c_reg = ClassicalRegister(2, name='c')

# Combine them into a Quantum Circuit
qc_exp3 = QuantumCircuit(q_reg, c_reg)

print("Circuit created with", qc_exp3.num_qubits, "qubits and", qc_exp3.num_clbits, "classical bits.")
display(qc_exp3.draw('mpl'))
```

Circuit created with 2 qubits and 2 classical bits.



```
# =====
# Experiment No: 4
# Objective: Design and simulate a basic quantum circuit using Qiskit.
# =====

from qiskit import QuantumCircuit

# Create a simple circuit with exactly 1 Qubit
qc_exp4 = QuantumCircuit(1)

# Apply a Pauli-X gate (equivalent to a classical NOT gate)
# This changes the qubit from default state |0> to |1>
qc_exp4.x(0)

print("Simple quantum circuit designed:")
display(qc_exp4.draw('mpl'))
```

Simple quantum circuit designed:



```
# =====
# Experiment No: 5
# Objective: Execute a basic "Hello Quantum World" program
# by combining gates and measurements, and analyze the output.
# =====

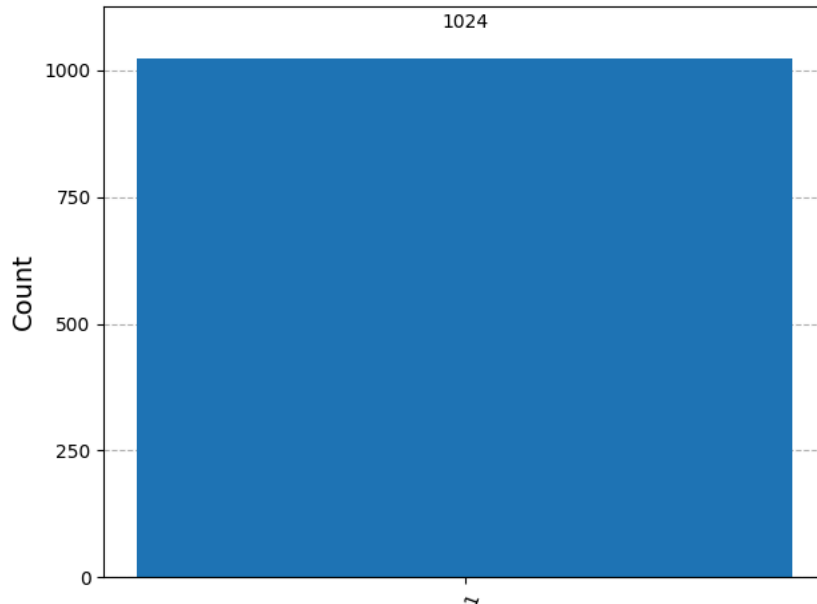
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# 1. Build the circuit (1 Qubit, 1 Classical bit)
qc_exp5 = QuantumCircuit(1, 1)
qc_exp5.x(0)          # Flip 0 to 1
qc_exp5.measure(0, 0) # Measure the qubit and store result in the classical bit

# 2. Simulate the circuit
simulator = AerSimulator()
job = simulator.run(qc_exp5, shots=1024)
result = job.result()
counts = result.get_counts()

# 3. Analyze output
print("Hello Quantum World! Measurement outcomes:", counts)
display(plot_histogram(counts))
```

Hello Quantum World! Measurement outcomes: {'1': 1024}



```
# =====
# Experiment No: 6
# Objective: Apply single-qubit gates (Pauli-X, Pauli-Y, Pauli-Z)
# and observe how they are placed in the circuit for state transformations.
# =====

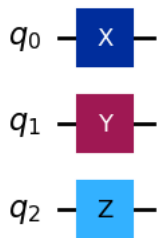
from qiskit import QuantumCircuit

# Create a 3-qubit circuit to demonstrate 3 different gates simultaneously
qc_exp6 = QuantumCircuit(3)

qc_exp6.x(0) # Applies Pauli-X (Bit-flip) on Qubit 0
qc_exp6.y(1) # Applies Pauli-Y (Bit and Phase-flip) on Qubit 1
qc_exp6.z(2) # Applies Pauli-Z (Phase-flip) on Qubit 2

print("Circuit with Single-Qubit Pauli Gates:")
display(qc_exp6.draw('mpl'))
```

Circuit with Single-Qubit Pauli Gates:



```
# =====
# Experiment No: 7 & 8
# Objective: Generate superposition states using Hadamard gates
# and visualize the resulting qubit states on the Bloch sphere.
# =====

from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector
from qiskit.visualization import plot_bloch_multivector

# Create a 1-qubit circuit
qc_exp7_8 = QuantumCircuit(1)

# Apply Hadamard gate to create perfect superposition
qc_exp7_8.h(0)

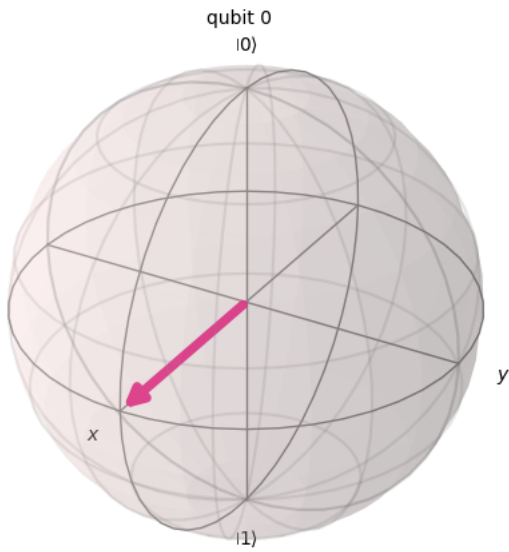
display(qc_exp7_8.draw('mpl'))

# Extract the state vector BEFORE measurement to visualize it
state = Statevector.from_instruction(qc_exp7_8)
```

```
print("Visualizing Superposition: Notice the state vector pointing to the equator of the Bloch Sphere.")
display(plot_bloch_multivector(state))
```

q — **H** —

Visualizing Superposition: Notice the state vector pointing to the equator of the Bloch Sphere.



```
# =====
# Experiment No: 9
# Objective: Measure qubits in superposition and analyze
# the resulting probabilistic outcomes and distributions.
# =====

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

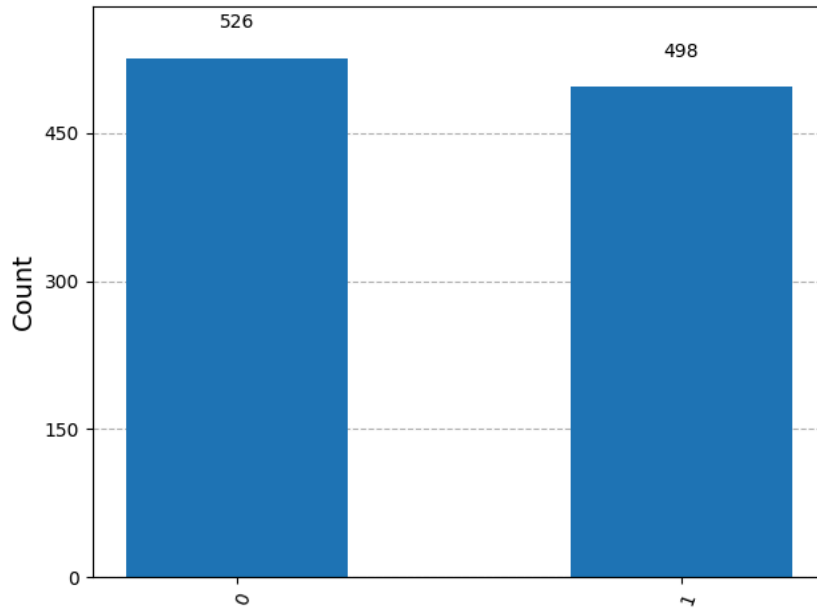
# Create circuit with 1 qubit and 1 classical bit
qc_exp9 = QuantumCircuit(1, 1)

qc_exp9.h(0)          # Put qubit into superposition
qc_exp9.measure(0, 0) # Measure the collapsed state

# Simulate running the circuit 1024 times
simulator = AerSimulator()
counts = simulator.run(qc_exp9, shots=1024).result().get_counts()

print("Probability Distribution: Because of superposition, outcomes are roughly 50% for '0' and 50% for '1'.")
display(plot_histogram(counts))
```

Probability Distribution: Because of superposition, outcomes are roughly 50% for '0' and 50% for '1'.



```
# =====
# Experiment No: 10 & 11
# Objective: Implement Bell states using Hadamard and CNOT gates to
# understand entanglement and verify the correlated non-classical measurement results.
# =====

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# Create a circuit with 2 Qubits and 2 Classical bits
qc_exp10_11 = QuantumCircuit(2, 2)

# Step 1: Put the Control Qubit (Qubit 0) into superposition
qc_exp10_11.h(0)

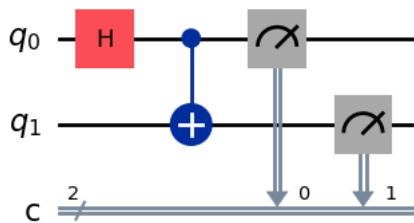
# Step 2: Apply CNOT gate. Target Qubit (1) flips ONLY if Control Qubit (0) is 1.
# This creates the entangled Bell State.
qc_exp10_11.cx(0, 1)

# Measure both qubits
qc_exp10_11.measure([0, 1], [0, 1])

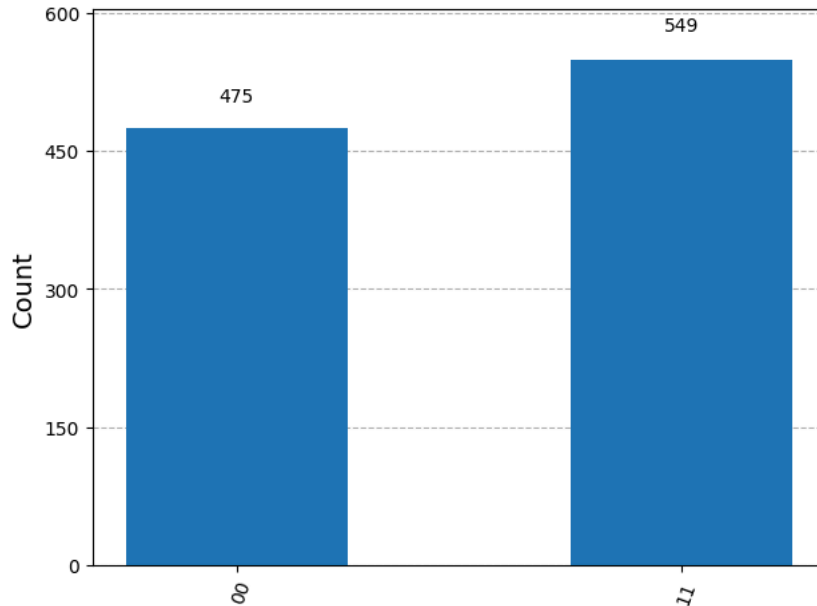
display(qc_exp10_11.draw('mpl'))

# Simulate
simulator = AerSimulator()
counts = simulator.run(qc_exp10_11, shots=1024).result().get_counts()

print("Entangled Results: We ONLY measure perfectly correlated states ('00' or '11').")
display(plot_histogram(counts))
```



Entangled Results: We ONLY measure perfectly correlated states ('00' or '11').



```
# =====
# Experiment No: 10 & 11
# Objective: Implement Bell states using Hadamard and CNOT gates to
# understand entanglement and verify the correlated non-classical measurement results.
# =====

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# Create a circuit with 2 Qubits and 2 Classical bits
qc_exp10_11 = QuantumCircuit(2, 2)

# Step 1: Put the Control Qubit (Qubit 0) into superposition
qc_exp10_11.h(0)

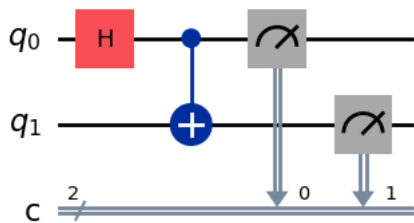
# Step 2: Apply CNOT gate. Target Qubit (1) flips ONLY if Control Qubit (0) is 1.
# This creates the entangled Bell State.
qc_exp10_11.cx(0, 1)

# Measure both qubits
qc_exp10_11.measure([0, 1], [0, 1])

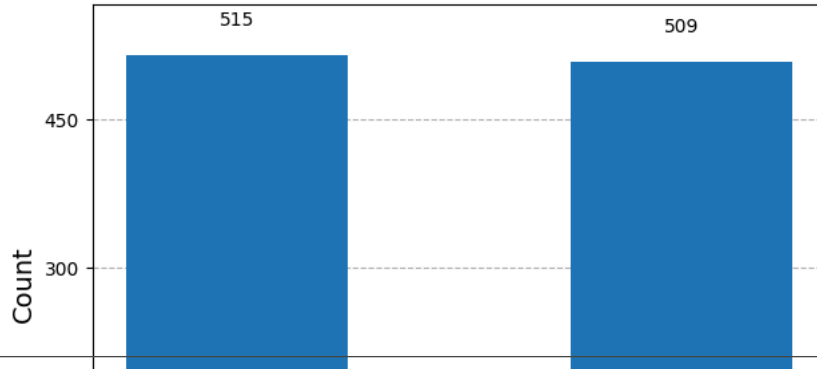
display(qc_exp10_11.draw('mpl'))

# Simulate
simulator = AerSimulator()
counts = simulator.run(qc_exp10_11, shots=1024).result().get_counts()

print("Entangled Results: We ONLY measure perfectly correlated states ('00' or '11').")
display(plot_histogram(counts))
```



Entangled Results: We ONLY measure perfectly correlated states ('00' or '11').



```
# =====
# Experiment No: 12, 13, & 14
# Objective: Design and execute the Deutsch-Jozsa algorithm using Qiskit to
# understand constant and balanced functions, analyze the results,
# and demonstrate quantum advantage (solving it in exactly one query).
# =====

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# 1. CREATE THE ORACLE (The "Hidden Function")
n = 3 # We will use 3 input qubits

# Let's build a "Balanced" Oracle (returns half 0s, half 1s)
oracle = QuantumCircuit(n+1)
for qubit in range(n):
    oracle.cx(qubit, n) # Entangle the inputs with the target output qubit

# 2. BUILD THE MAIN ALGORITHM CIRCUIT
dj_circuit = QuantumCircuit(n+1, n)

# Step A: Apply Hadamard gates to all input qubits
for qubit in range(n):
    dj_circuit.h(qubit)

# Step B: Put target qubit in the |-> state (required for phase kickback)
dj_circuit.x(n)
dj_circuit.h(n)
dj_circuit.barrier() # Visual separator

# Step C: Add our Oracle to the circuit
dj_circuit = dj_circuit.compose(oracle)
dj_circuit.barrier() # Visual separator

# Step D: Apply Hadamard gates to input qubits again and measure them
for qubit in range(n):
    dj_circuit.h(qubit)
    dj_circuit.measure(qubit, qubit)

print("Full Deutsch-Jozsa Circuit Implementation:")
display(dj_circuit.draw('mpl'))

# 3. ANALYZE AND DEMONSTRATE QUANTUM ADVANTAGE
```